



UNIDADE I

Desenvolvimento Web com .NET

Prof. Dr. Tarcísio Peres

O que será visto na disciplina

- Pipeline HTTP e endpoints;
- MVC e Razor;
- Persistência e conexão com BD com EF Core;
- APIs REST e OpenAPI;
- Integração React/Node – Consumo da API;
- Autenticação e autorização ponta a ponta;
- Observabilidade e resiliência;
- Build, implantação e CI/CD full-stack.

Aplicação web conecta cliente e servidor

- Cliente: navegador, aplicativo móvel ou outro sistema inicia a interação.
- Requisição: pede um recurso ou solicita uma operação ao servidor.
- Servidor: recebe o pedido, interpreta a intenção e escolhe o código adequado.
- Processamento: aplica regras, consulta dados e prepara o resultado.
- Resposta: devolve status, cabeçalhos e conteúdo ao cliente.
- Exemplo: no PetMatch, GET /api/pets retorna a lista de animais em JSON.

HTTP organiza a conversa

- Requisição: reúne método, URL, cabeçalhos e, quando necessário, corpo.
 - Método: indica a operação pretendida, como GET ou POST.
 - URL: identifica a rota e pode incluir valores de rota ou de consulta.
 - Cabeçalhos: descrevem metadados, como o formato enviado ou aceito.
 - Corpo: transporta dados estruturados para o servidor, quando aplicável.
- Resposta: contém status, cabeçalhos e o conteúdo devolvido ao cliente.

O pipeline conduz cada requisição

- Pipeline HTTP: é o caminho interno entre a chegada da requisição e a resposta.
- Etapas: formam uma sequência organizada de responsabilidades.
- Objetivo: preparar o ambiente, decidir o destino e produzir uma saída coerente.
- ASP.NET Core: configura esse percurso na inicialização da aplicação.
- Cadeia: cada componente pode atuar e encaminhar o processamento.
- Destino: o endpoint selecionado encerra o percurso com uma resposta.

Middleware é uma etapa intermediária

- Middleware: é um componente que participa da requisição e da resposta.
 - Examinar: pode ler informações antes de decidir o que fazer.
 - Modificar: pode ajustar a requisição ou a resposta.
 - Encaminhar: pode chamar a próxima etapa do pipeline.
 - Interromper: pode produzir uma resposta sem continuar a cadeia.
- Uso transversal: erros, HTTPS, arquivos estáticos, autenticação e roteamento.

A ordem dos middlewares muda o resultado

- Regra central: a sequência configurada faz parte do comportamento da aplicação.
 1. UseHttpsRedirection: redireciona o tráfego para HTTPS quando aplicável.
 2. UseDefaultFiles: associa a raiz a um arquivo inicial em wwwroot.
 3. UseStaticFiles: entrega HTML, CSS e JavaScript ao navegador.
 4. MapGet e MapPost: publicam *endpoints* para as chamadas da API.
- Consequência: uma etapa anterior pode atender à requisição antes das seguintes.

Interatividade

Em uma aplicação ASP.NET Core, qual alternativa descreve melhor o papel do middleware?

- a) É sempre o código final que gera a resposta da API.
- b) É um componente do pipeline que pode processar a requisição e a resposta.
- c) É uma classe obrigatória para representar dados do banco.
- d) É o arquivo HTML inicial da aplicação.
- e) É o mecanismo usado apenas para criar controllers.

Resposta

Em uma aplicação ASP.NET Core, qual alternativa descreve melhor o papel do middleware?

- a) É sempre o código final que gera a resposta da API.
- b) É um componente do pipeline que pode processar a requisição e a resposta.**
- c) É uma classe obrigatória para representar dados do banco.
- d) É o arquivo HTML inicial da aplicação.
- e) É o mecanismo usado apenas para criar controllers.

Arquivos estáticos formam a interface

- `wwwroot`: concentra os arquivos públicos, como HTML, CSS, JavaScript e imagens.
- `UseDefaultFiles`: permite que `/` encontre automaticamente o arquivo inicial.
- `UseStaticFiles`: entrega o arquivo solicitado diretamente ao navegador.
- No cliente: o navegador interpreta o HTML, aplica o CSS e executa o JavaScript.
- Separação: arquivos estáticos compõem a tela; endpoints fornecem dados dinâmicos.
- `PetMatch`: o mesmo servidor entrega a página e responde à API em JSON.

Roteamento associa URLs a ações

- Roteamento: relaciona padrões de endereço a destinos executáveis.
- Leitura da URL: identifica trechos fixos, valores variáveis e restrições.
 - Exemplo 1: `/api/pets` representa a listagem de animais.
 - Exemplo 2: `/api/pets/{id:int}` captura um identificador inteiro.
- Benefício: evita comparações manuais de endereços espalhadas pelo código.
- Resultado: método e rota compatíveis conduzem ao endpoint adequado.

Endpoint combina método, rota e função

- Endpoint: é a associação entre método HTTP, rota e função de tratamento.
 - GET /api/pets: consulta e devolve os animais disponíveis.
 - GET /api/pets/2: consulta o animal identificado por 2.
 - POST /api/interesses: recebe uma manifestação de interesse em adoção.
- Entrada: o binding fornece parâmetros e objetos tipados à função.
- Saída: a função devolve status e conteúdo, normalmente em JSON.

Minimal APIs deixam o fluxo explícito

- Declaração direta: MapGet e MapPost aproximam a rota da função executada.
- Clareza inicial: o caminho entre requisição e resposta fica fácil de observar.
- Boas aplicações: aprendizagem, demonstrações e APIs com poucos endpoints.
- Menos estrutura: não exige controllers antes de existir essa necessidade.
- Crescimento: grupos de rotas e métodos de extensão melhoram a organização.
- Atenção: se Program.cs acumular muitos fluxos, extraia endpoints ou avalie MVC.

GET consulta dados

- Finalidade: realizar leitura ou consulta de um recurso.
 - Sem filtros: GET /api/pets solicita a lista disponível.
 - Por rota: GET /api/pets/1 solicita um item específico.
 - Por query string: GET /api/pets?especie=Gato refina a listagem.
- Binding: valores da rota ou da consulta chegam como parâmetros C#.
- Resposta: o navegador pode receber diretamente o status e o JSON produzido.

POST envia dados para processamento

- Finalidade: enviar dados relacionados a uma ação do usuário.
- Requisição: combina o método POST com a rota de destino.
- Cabeçalho: Content-Type: application/json informa o formato do corpo.
- Corpo: `JSON.stringify` converte o objeto JavaScript em JSON.
- Binding: o ASP.NET Core associa o JSON a um objeto C#.
- Resultado: o endpoint valida e pode devolver sucesso, erro de entrada ou ausência.

Binding de rota extrai valores da URL

- Origem: o valor ocupa um segmento definido na rota.
- Modelo: `/api/pets/{id:int}` declara um parâmetro inteiro chamado `id`.
- Chamada: `/api/pets/3` fornece o valor `3` para esse parâmetro.
- Conversão: o ASP.NET Core entrega o valor ao handler como `int id`.
- Restrição: `:int` exige um segmento compatível com número inteiro.
- Texto: `/api/boas-vindas/{nome}` fornece uma string chamada `nome`.

Binding de query string aplica filtros

- Origem: os pares aparecem na URL depois do caractere ?.
- Formato: chave=valor; parâmetros adicionais são separados por &.
- Exemplo: /api/pets?especie=Gato&cidade=Curitiba.
- Uso: filtros opcionais refinam uma listagem sem mudar a rota.
- Binding: nomes como **especie**, porte e cidade chegam aos parâmetros C#.
- Ausência: um parâmetro omitido permanece nulo e não restringe a consulta.

Binding de corpo converte JSON em C#

- Origem: os dados chegam no corpo da requisição HTTP.
- Quando usar: uma ação reúne vários valores relacionados.
- JavaScript: cria um objeto com petId, nome, e-mail e mensagem.
- Serialização: `JSON.stringify` transforma o objeto em texto JSON.
- Formato: Content-Type: `application/json` orienta a leitura do corpo.
- Binding: o ASP.NET Core cria `AdoptionInterestRequest` para validação.

Minimal APIs e MVC atendem a escalas diferentes

- Minimal APIs: endpoints diretos e pouca estrutura obrigatória.
- Melhor encaixe: poucos endpoints, regras localizadas e consumo direto de JSON.
- Organização: grupos e arquivos próprios preservam clareza conforme o código cresce.
- MVC: controllers, actions, modelos, views, filtros e convenções formais.
- Melhor encaixe MVC: muitos fluxos, telas no servidor e regras reutilizáveis.
- Escolha: considere escopo, clareza, evolução e testabilidade do sistema.

Do navegador à interface atualizada

1. Página: o navegador acessa / e inicia uma requisição HTTP.
2. Arquivos: os middlewares entregam index.html, CSS e JavaScript.
3. Chamada: o JavaScript usa fetch para consultar um endpoint da API.
4. Pipeline: middlewares e roteamento selecionam o destino adequado.
5. Processamento: binding prepara a entrada; o endpoint devolve status e JSON.
6. Interface: o JavaScript lê a resposta e atualiza cards ou mensagens.

Interatividade

Considere o endpoint:

```
app.MapGet("/api/produtos/{id:int}", (int id) =>
{
    return Results.Ok(id);
});
```

De onde vem o valor de id?

- a) Do corpo JSON da requisição.
- b) Do cabeçalho HTTP.
- c) Da query string.
- d) Da rota acessada pelo navegador.
- e) De um arquivo estático.

Resposta

Considere o endpoint:

```
app.MapGet("/api/produtos/{id:int}", (int id) =>
{
    return Results.Ok(id);
});
```

De onde vem o valor de id?

- a) Do corpo JSON da requisição.
- b) Do cabeçalho HTTP.
- c) Da query string.
- d) Da rota acessada pelo navegador.
- e) De um arquivo estático.

MVC separa responsabilidades

- Aplicações maiores reúnem páginas, formulários, validações e navegação.
- Sem separação, regras, decisões de fluxo e HTML ficam misturados.
- A entrada identifica a ação solicitada pelo usuário.
- O processamento coordena dados, validações e decisões da aplicação.
- A apresentação transforma o resultado em uma página compreensível.
- A divisão facilita localizar mudanças, testar partes e manter o sistema.

O modelo MVC distribui papéis claros

- Model representa dados e conceitos do domínio da aplicação.
- View constrói a interface e apresenta dados, estados e mensagens.
- Controller recebe a requisição e coordena o processamento necessário.
- O controller escolhe a View ou outro tipo de resposta adequado.
- Cada parte mantém um papel definido no fluxo entre usuário e servidor.
- No PetMatch, PetsController prepara dados para as Views de pets.

Razor combina HTML e C# com controle

- Uma Razor View usa a extensão .cshtml e preserva a estrutura do HTML.
- @model declara o tipo de dado recebido pela página.
- @Model permite exibir propriedades preparadas pelo controller.
- @if controla trechos exibidos conforme o estado da aplicação.
- @foreach percorre coleções e produz elementos repetidos da interface.
- Razor apresenta dados; regras de negócio permanecem fora da View.

Controllers agrupam actions relacionadas

- Controller é uma classe que reúne operações de uma mesma área.
- Action é um método que responde a uma requisição compatível.
- Index com GET consulta dados e entrega a coleção para a View.
- Create com GET apresenta um formulário vazio ao usuário.
- Create com POST recebe o ViewModel preenchido pelo model binding.
- A action coordena o fluxo sem concentrar toda a lógica do sistema.

MVC precisa ser registrado e roteado

- `AddControllersWithViews()` registra controllers e Razor Views.
- `MapControllerRoute(...)` publica uma rota convencional para o MVC.
- O padrão `{controller}/{action=Index}/{id?}` define valores esperados.
- `/Pets` pode localizar `PetsController` e executar a action `Index`.
- `MapPetEndpoints()` mantém os endpoints Minimal API já existentes.
- Arquivos estáticos, API e páginas MVC convivem no mesmo projeto.

ViewModel representa uma tela

- ViewModel descreve os dados esperados por uma página ou formulário.
- Ele não precisa repetir todas as propriedades do modelo principal.
- Inclui somente os campos necessários para a interação atual.
- PetCreateViewModel reúne os valores usados no cadastro de um pet.
- O model binding preenche o ViewModel com os dados enviados.
- Após a validação, o controller converte a entrada em dados do domínio.

Interatividade

Em uma aplicação ASP.NET Core MVC, qual é a principal função de um ViewModel?

- a) Substituir o controller.
- b) Representar os dados específicos de uma tela ou formulário.
- c) Renderizar diretamente o HTML final.
- d) Configurar o pipeline HTTP.
- e) Criar automaticamente o banco de dados.

Resposta

Em uma aplicação ASP.NET Core MVC, qual é a principal função de um ViewModel?

- a) Substituir o controller.
- b) Representar os dados específicos de uma tela ou formulário.**
- c) Renderizar diretamente o HTML final.
- d) Configurar o pipeline HTTP.
- e) Criar automaticamente o banco de dados.

Data Annotations declaram regras

- Atributos são colocados nas propriedades do ViewModel.
- [Required] exige o preenchimento do campo.
- [StringLength] define limites mínimo e máximo de texto.
- [Range] restringe valores numéricos a uma faixa aceita.
- [Display] define o nome apresentado ao usuário.
- As mensagens configuradas aparecem na View por meio de Tag Helpers.

ModelState controla o fluxo de validação

- O MVC preenche o ViewModel e avalia as Data Annotations.
- ModelState.IsValid informa se a entrada atende às regras declaradas.
- Se for inválido, a action retorna View(viewModel).
- A mesma página preserva os valores e mostra as mensagens de erro.
- Se for válido, a action processa os dados e registra o resultado.
- Depois, RedirectToAction leva o usuário à listagem atualizada.

Filtros tratam comportamentos comuns

- Filtros executam lógica antes ou depois de uma action.
- Eles centralizam comportamentos que seriam repetidos em vários pontos.
- No PetMatch, um filtro adiciona informação contextual em ViewData.
- Outros usos incluem logs, políticas e tratamento padronizado de erros.
- O filtro participa do fluxo MVC sem substituir o controller.
- Também não substitui validações do formulário ou regras de negócio.

Layouts definem a estrutura compartilhada

- `_Layout.cshtml` concentra a moldura comum das páginas Razor.
- Cabeçalho, navegação, CSS, área principal e rodapé ficam centralizados.
- `_ViewStart.cshtml` indica qual layout será usado pelas Views.
- `@RenderBody()` marca o ponto em que o conteúdo da View será inserido.
- Cada View fica responsável apenas pelo conteúdo específico da página.
- Uma mudança no layout atualiza todas as páginas que o utilizam.

Partials evitam repetição de HTML

- Partial é um fragmento reutilizável de apresentação em arquivo .cshtml.
- Ele não substitui controllers ou actions.
- Um partial tipado pode receber um Model próprio para renderização.
- `_PetCard.cshtml` concentra o HTML de cada card de pet.
- `<partial name="_PetCard" model="pet" />` reutiliza o fragmento.
- Alterar o partial atualiza todos os locais que usam esse bloco.

Tag Helpers conectam HTML ao MVC

- Tag Helpers são atributos e elementos processados pelo ASP.NET Core.
- `asp-for` liga campos, rótulos e valores às propriedades do ViewModel.
- `asp-validation-for` mostra a mensagem do campo correspondente.
- `asp-action` e `asp-controller` geram formulários e links conforme as rotas.
- `asp-validation-summary` reúne os erros da validação em um único ponto.
- `_ViewImports.cshtml` habilita esses recursos para todas as Views.

Interatividade

No MVC, quando uma action POST recebe dados inválidos de um formulário, qual prática é mais adequada?

- a) Ignorar os erros e salvar os dados.
- b) Redirecionar imediatamente para a página inicial.
- c) Verificar ModelState.IsValid e devolver a mesma View com mensagens.
- d) Apagar o ViewModel.
- e) Trocar a action POST por GET.

Resposta

No MVC, quando uma action POST recebe dados inválidos de um formulário, qual prática é mais adequada?

- a) Ignorar os erros e salvar os dados.
- b) Redirecionar imediatamente para a página inicial.
- c) Verificar `ModelState.IsValid` e devolver a mesma View com mensagens.
- d) Apagar o ViewModel.
- e) Trocar a action POST por GET.

Referências

- BARTH, A. *HTTP State Management Mechanism*. RFC Editor, 2011. RFC 6265. Disponível em: <https://www.rfc-editor.org/info/rfc6265>. Acesso em: 6 jul. 2026.
- BRIND, M. *ASP.NET Core Razor Pages in Action*. Shelter Island: Manning, 2022.
- DELAMATER, M.; MURACH, J. *Murach's ASP.NET Core MVC*. 2. ed. Fresno: Mike Murach & Associates, 2022.
- FREEMAN, A. *Pro ASP.NET Core 7*. 10. ed. Shelter Island: Manning, 2023.
- LOCK, A. *ASP.NET Core in Action*. 2. ed. Shelter Island: Manning, 2021.

ATÉ A PRÓXIMA!